LINFO2132 - Languages and Translators

Final Project Report: Compiler Implementation

Louis CAYPHAS - 49862200
Pierre-Jean BOREUX - 58512200

May 2026

Contents

1 Introduction

This project consists in implementing a complete compiler pipeline in Java for the language defined in the LINFO2132 project statement.

The compiler includes:

- a handwritten lexer,
- a recursive-descent parser generating an AST,
- a semantic analyser performing type and scope verification,
- a JVM bytecode generator based on ASM.

In addition to the required features, we implemented several extensions such as utility built-in functions, implicit `INT` to `FLOAT` promotion, flexible for-loop iterators, and extended typed-array support.

This report presents the compiler architecture, implementation choices, semantic verification, code generation, testing strategy, and final grammar.

2 Compiler Overview

The compiler follows the standard compilation pipeline:

`Source Code` → `Lexer` → `Parser / AST` → `Semantic Analysis` → `JVM Bytecode`

The project is organised into four main packages:

- `compiler.Lexer`,
- `compiler.parser`,
- `compiler.Semantic_Analysis`,
- `compiler.codegen`.

The compiler entry point is implemented in `Compiler.java`. The `test` directory contains JUnit tests for all compilation phases.

2.1 Project Structure

The project follows the Gradle structure provided in the project skeleton. The source code is organised into several packages corresponding to the main phases of the compiler:

- `compiler.Lexer` contains the lexer and token representation,
- `compiler.parser` contains the recursive-descent parser and AST nodes,
- `compiler.Semantic_Analysis` contains semantic verification,
- `compiler.codegen` contains JVM bytecode generation using ASM.

The `test` directory contains JUnit tests and example source programs separated according to the different compilation phases: lexer, parser, semantic analysis, and code generation.

3 Lexer

The lexer was implemented manually using a character-by-character approach. It recognises:

- keywords,
- primitive types,
- literals,
- identifiers and collection names,
- operators and punctuation symbols.

Whitespace and comments beginning with `#` are ignored.

Lexical errors such as unterminated strings, malformed operators, or unexpected characters produce runtime exceptions and terminate the compiler with a non-zero exit code.

Dedicated JUnit tests verify valid and invalid token sequences, comments, strings, operators, and numeric literals.

4 Parser

The parser is implemented as a handwritten recursive-descent parser producing an Abstract Syntax Tree (AST).

It supports:

- variable and function declarations,
- collections,
- assignments,
- conditionals and loops,
- function calls,
- arrays and field accesses.

Expression parsing is separated into precedence levels:

$$|| \rightarrow \&\& \rightarrow == \rightarrow <, >, <=, >= \rightarrow +, - \rightarrow *, /, \%$$

The parser generates dedicated AST nodes such as `ProgramNode`, `FuncDeclNode`, `BinaryOpNode`, `IfNode`, `ForNode`, and `ArrayAccessNode`.

Syntax errors produce runtime exceptions with descriptive messages.

The complete grammar is provided in the appendix.

5 Semantic Analysis

Semantic analysis traverses the AST using a scoped symbol table.

The analyser verifies:

- variable and function declarations,
- scope resolution,
- assignments,
- operators,
- return statements,
- arrays and collections,
- function calls and constructor arguments.

Lexical scoping is implemented using a stack of scopes. Final variables cannot be reassigned.

The analyser reports all required semantic errors: `TypeError`, `CollectionError`, `OperatorError`, `ArgumentError`, `MissingConditionError`, `ReturnError`, and `ScopeError`.

6 Code Generation

The final compiler phase generates JVM bytecode using ASM.

Functions are translated into static JVM methods, while global variables are generated as static fields of the main class. Collection declarations generate separate JVM classes with fields and constructors.

The generator supports:

- functions and returns,
- local and global variables,
- arrays,
- collections,
- built-in functions,
- arithmetic and logical expressions.

Primitive arrays use `NEWARRAY`, while reference arrays use `ANEWARRAY`. Built-in functions are compiled directly to JVM instructions or Java standard library calls.

Generated programs are written as `.class` files and executed directly on the JVM.

7 Extra Features

The project statement already requires basic support for functions, collections, arrays, scopes, and standard input/output. Therefore, we only list here the features that go beyond this minimal expected behaviour or that extend it.

7.1 Utility Built-in Functions

In addition to the standard input/output built-ins, we implemented several utility functions:

- `str(INT)`, which converts an integer to a string,
- `floor(FLOAT)`, which returns the greatest integer less than or equal to the given float,
- `ceil(FLOAT)`, which returns the smallest integer greater than or equal to the given float,
- `length(STRING or ARRAY)`, which returns the length of a string or array.

These functions are registered during semantic analysis and compiled directly to JVM instructions or Java standard library calls.

7.2 INT to FLOAT Promotion

We added implicit promotion from `INT` to `FLOAT` in compatible contexts. This is checked during semantic analysis and handled during code generation by emitting the JVM `I2F` instruction when needed.

For example, a function declared as returning `FLOAT` can return an `INT` expression, which is automatically converted in the generated bytecode.

7.3 Flexible For-Loop Iterator

The parser supports two forms of for-loop iterator. The iterator can either be declared directly inside the loop or be an already declared variable:

- `for (INT i; 1 -> 10; i + 1)`
- `for (i; 1 -> 10; i + 1)`

This required the loop iterator to be represented more generally as an AST node, instead of only supporting variable declarations.

7.4 Extended Typed Arrays

Although one-dimensional arrays are part of the base language, we extended their code generation to support several element types. The code generator handles arrays of `INT`, `FLOAT`, `BOOL`, and reference types such as `STRING`.

Primitive arrays are generated with `NEWARRAY`, while reference arrays are generated with `ANEWARRAY`. This was tested with array allocation, indexed assignment, and indexed access.

8 Testing

The project contains 59 JUnit tests covering all compilation phases.

Tests verify:

- token recognition and lexical errors,
- AST construction and syntax errors,
- semantic verification and required error categories,
- JVM bytecode generation and execution.

Code-generation tests automatically execute the generated `.class` files and compare their outputs with the expected results.

9 GitHub Repository

The complete source code of the project is available on GitHub:

`https://github.com/piboreux/projet_LINFO2132`

10 Use of AI Tools

Generative AI tools were used as support tools during the project development. Their use remained limited to assistance, debugging, verification, and report improvement tasks.

AI tools were mainly used for:

- debugging specific implementation issues, especially during parser, semantic analysis, and JVM bytecode generation development,
- clarifying technical details related to Java, ASM, Gradle, and JVM instructions,
- generating initial examples of JUnit tests that were later adapted and completed manually,
- checking consistency between the implementation and the project specifications,
- improving the wording, translation, and organisation of the report.

11 Conclusion

This project allowed us to implement a complete compiler pipeline in Java, from lexical analysis to JVM bytecode generation.

The final compiler supports the main language features required by the project statement, including functions, collections, arrays, semantic verification, and executable JVM bytecode generation using ASM.

The project also provided practical experience with compiler construction, AST manipulation, semantic analysis, scoped symbol tables, and low-level JVM bytecode generation.

12 Grammar

The following grammar corresponds to the final version supported by our parser.

Program	::=	{ Statement } EOF
Statement	::=	VarDecl | FuncDecl | CollDecl | IfStmt | WhileStmt | ForStmt | ReturnStmt | Assignment | CallStmt | Block
VarDecl	::=	["final"] Type identifier ["=" Expression] ";"
Type	::=	TypeName ["[]"]
TypeName	::=	"INT" | "FLOAT" | "BOOL" | "STRING" | CollectionName
FuncDecl	::=	"def" [Type] identifier "(" [ParamList] ")" Block
ParamList	::=	Param { "," Param }
Param	::=	Type identifier
CollDecl	::=	"coll" CollectionName "{" { FieldDecl } "}"
FieldDecl	::=	Type identifier ";"
Assignment	::=	AssignTarget "=" Expression ";"
AssignTarget	::=	identifier | AssignTarget "[" Expression "]" | AssignTarget "." identifier
CallStmt	::=	identifier "(" [ArgList] ")" ";"
IfStmt	::=	"if" "(" Expression ")" Block ["else" (IfStmt | Block)]
WhileStmt	::=	"while" "(" Expression ")" Block
ForStmt	::=	"for" "(" (Type identifier | identifier) ";" Expression "->" Expression ";" Expression ")" Block
ReturnStmt	::=	"return" [Expression] ";"
Block	::=	"{" { Statement } "}"
Expression	::=	OrExpr
OrExpr	::=	AndExpr { "||" AndExpr }

AndExpr	::=	EqualityExpr { "&&" EqualityExpr }
EqualityExpr	::=	ComparisonExpr { ("==" | "=/=") ComparisonExpr }
ComparisonExpr	::=	AdditionExpr { ("<" | ">" | "<=" | ">=") AdditionExpr }
AdditionExpr	::=	MultiplicationExpr { ("+" | "-") MultiplicationExpr }
MultiplicationExpr	::=	UnaryExpr { ("*" | "/" | "%") UnaryExpr }
UnaryExpr	::=	("not" | "-") UnaryExpr | PostfixExpr
PostfixExpr	::=	PrimaryExpr { "[" Expression "]" | "." identifier }
PrimaryExpr	::=	Literal | identifier | identifier "(" [ArgList] ")" | CollectionName "(" [ArgList] ")" | ArrayConstructor | "(" Expression ")"
ArrayConstructor	::=	ArrayBaseType "ARRAY" "[" Expression "]"
ArrayBaseType	::=	"INT" | "FLOAT" | "BOOL" | "STRING"
ArgList	::=	Expression { "," Expression }
Literal	::=	IntegerLiteral | FloatLiteral | StringLiteral | BooleanLiteral